

# UART Transceiver (UART\_TX & UART\_RX)

## SystemVerilog RTL Design Documentation

Digital Design Report by: Yassin Kamel

September 2026

### Contents

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>2</b> |
| <b>2</b> | <b>RTL Implementation: UART Transmitter (UART_TX)</b> | <b>2</b> |
| 2.1      | FSM Controller . . . . .                              | 2        |
| 2.2      | Parity Calculator . . . . .                           | 2        |
| 2.3      | Serialiser . . . . .                                  | 3        |
| 2.4      | Multiplexer . . . . .                                 | 3        |
| 2.5      | Top-Level Transmitter Integration . . . . .           | 3        |
| <b>3</b> | <b>RTL Implementation: UART Receiver (UART_RX)</b>    | <b>4</b> |
| 3.1      | Start Bit Edge Detector . . . . .                     | 4        |
| 3.2      | FSM Controller . . . . .                              | 4        |
| 3.3      | Deserialiser . . . . .                                | 4        |
| 3.4      | Parity Verification Integration . . . . .             | 5        |
| 3.5      | Output Logic and Error Handling . . . . .             | 5        |
| 3.6      | Top-Level Receiver Integration . . . . .              | 5        |

# 1 Introduction

This document is a thorough documentation for the UART TX and RX modules project. The TX module is composed of an FSM controller, a parity calculator, a serialiser and a multiplexer which handles the output of these modules based on the controller. While the RX module consists of an FSM controller as well, the same parity calculator used by the TX, a deserialiser, an edge detector and an output logic unit.

## 2 RTL Implementation: UART Transmitter (UART\_TX)

### 2.1 FSM Controller

The transmitter controller manages the operational states (**free**, **start**, **data**, **parity**, and **stop**) which represent which state the data transmission has reached. Using these states, the module is able to control the rest of the modules properly and in time.

Default assignments are placed at the beginning of the combinational block to avoid unintended latch creation. Control signals are grouped within a packed struct (`uart_tx_ctrl_t`) containing the multiplexer select lines (`mux_sel`), shift enable (`shift_en`), load pulse (`load`), and busy flag (`busy`):

- **free**: The idle state. The line rests high, and `ctrl.busy` is lowered. When a valid input strobe (`v_input`) is asserted, `ctrl.load` is raised to latch the parallel data, and the FSM transitions to **start**.
- **start**: Drives the start bit onto the line (`mux_sel = 2'b00`, selecting 0). It transitions unconditionally to **data** on the next clock edge.
- **data**: Asserts `ctrl.shift_en` and directs the serializer output to the pin (`mux_sel = 2'b01`). When all bits are shifted out (`data_done`), it branches to **parity** if parity is enabled (`p_en`) or directly to **stop** if disabled.
- **parity**: Directs the calculated parity bit to the line (`mux_sel = 2'b10`) and transitions to **stop**.
- **stop**: Transmits the stop bit (`mux_sel = 2'b11`, selecting 1) and transitions back to **free**, completing the frame cycle.

### 2.2 Parity Calculator

This module calculates the parity bit in real time using combinational logic. It employs the bitwise XOR reduction operator (`^p_data`) across the parallel input vector and XORs the reduction result with the parity configuration bit (`i_par_odd`).

When `i_par_odd = 0` (even parity), the output equals `^p_data`, guaranteeing that the total count of ones including parity is even. When `i_par_odd = 1` (odd parity), the output is inverted, guaranteeing that the total count of ones is odd.

## 2.3 Serialiser

This module receives the parallel input and stores it into a shift-register, shifts the data with each clock cycle while wiring the LSB of the shift-register to the output and marks the done flag based on the counted shifted bits.

## 2.4 Multiplexer

This module implements a standard 4-to-1 multiplexer using an `always_comb` conditional tree. The multiplexer acts as the physical output selector for the transmitter, routing either framing bits, data bits, or parity to the TX line based on the 2-bit select code provided by the FSM controller.

## 2.5 Top-Level Transmitter Integration

The top-level transmitter module (`uart_tx`) instantiates and interconnects the controller, parity calculator, serializer, and multiplexer. Control signals are bundled into the `uart_tx_ctrl_t` bus (`ctrl_bus`), eliminating separate wiring clutter.

The 4-to-1 multiplexer inputs (`data[3:0]`) are mapped to:

- `data[0]` = `1'b0`: The active-low start bit, driven during the `start` state.
- `data[1]` = `serial_out_in`: The dynamic serialized data stream from the PISO register, driven during the `data` state.
- `data[2]` = `parity_out_in`: The dynamically computed parity bit, driven during the `parity` state.
- `data[3]` = `1'b1`: A constant logic high, shared between the idle (`free`) line condition and the transmission `stop` bit.

Because both the idle line state and the stop bit require a continuous logic high level, sharing input port `data[3]` allows the 5 distinct FSM states to be multiplexed onto the single-wire TX line using only a 2-bit select bus.

## 3 RTL Implementation: UART Receiver (UART\_RX)

### 3.1 Start Bit Edge Detector

The edge detector monitors transitions on the incoming serial line (**A**, connected to **i\_rx**). It maintains a registered copy of the line state from the previous clock cycle (**old**).

The module provides three edge detection outputs:

- **pos\_edge\_A** = **A** & ~**old**: Detects low-to-high (rising) edges.
- **neg\_edge\_A** = ~**A** & **old**: Detects high-to-low (falling) edges. This signal serves as the start bit trigger for the receiver FSM.
- **edge\_A** = **A** ^ **old**: Detects any transition on the serial line.

### 3.2 FSM Controller

The receiver controller orchestrates frame reception across four sequential states: **free**, **data**, **parity**, and **stop**. Unlike the TX FSM, it lacks a start state due to the FSM being triggered one clock cycle after the start has been detected by the edge detector module. Otherwise, its logic heavily follows the same TX FSM logic. It drives the packed control bus **uart\_rx\_ctrl\_t**, which contains **o\_busy**, **shift\_en**, **par\_state**, and **stop\_state**:

- **free**: The receiver sits idle with **ctrl.o\_busy** = 0. Upon detection of the start bit falling edge (**start\_bit\_edge**), the FSM advances to the **data** state on the subsequent clock cycle.
- **data**: Asserts **ctrl.shift\_en** to shift serial data bits into the deserializer. When **data\_done** is asserted by the deserializer bit counter, the FSM inspects the parity enable line (**i\_par\_en**): if enabled, it transitions to **parity**; otherwise, it skips directly to **stop**.
- **parity**: Asserts **ctrl.par\_state** = 1'b1, instructing the output logic to sample and latch the incoming parity bit. It transitions unconditionally to **stop**.
- **stop**: Asserts **ctrl.stop\_state** = 1'b1, prompting the output logic to inspect the stop bit level, verify frame and parity integrity, and strobe the valid flag. The FSM then returns to **free**.

### 3.3 Deserialiser

The deserializer converts incoming serial bits into a parallel bus of data using a shift register.

On each rising clock edge with **shift\_en** active, the serial bit **i\_rx** is concatenated to the MSB while existing bits are shifted right:

$$\text{parallel\_out} \leftarrow \{\text{i\_rx}, \text{parallel\_out}[\text{DATA\_W} - 1 : 1]\}$$

Because standard UART transmits the LSB first, this rightward shift ensures that after **DATA\_W** cycles, the first received bit reaches index 0 (**parallel\_out**[0]) while the final bit

sits at index `DATA_W - 1`. An internal counter (`count`) tracks received bits and asserts `data_done` when `count == DATA_W - 1`.

### 3.4 Parity Verification Integration

To verify data integrity without redundant logic, the receiver reuses the already initialised `parity_calc` module. As the deserializer accumulates the parallel data bus, `parity_calc` combinational logic calculates the `expected_parity` based on the received data payload and the configured mode (`i_par_odd`):

$$\text{expected\_parity} = (\text{\^data}) \oplus \text{i\_par\_odd}$$

This expected parity value is passed directly to the output logic, where it is evaluated against the bit actually transmitted over the serial channel.

### 3.5 Output Logic and Error Handling

The output logic module isolates the system outputs, outputs valid received data, and evaluates transmission error flags. It features a two-stage verification process synchronized to the FSM states:

1. **Parity Sampling:** During the `par_state` clock cycle, the module samples the physical line level and stores it in the internal flip-flop `actual_parity <= i_rx`.
2. **Stop Bit and Error Evaluation:** During the `stop_state` clock cycle, the module verifies that the stop bit is logic high (`i_rx == 1`):
  - If `stop_state && !i_rx`: The stop bit was received as 0 instead of 1. The module raises a framing error (`o_frame_err <= 1'b1`), latches the payload to `o_data`, asserts `o_valid <= 1'b1`, and flags a parity error if `actual_parity != expected_parity` while parity is enabled.
  - If `stop_state && i_rx`: The stop bit is valid. The module clears framing error (`o_frame_err <= 1'b0`), asserts `o_valid <= 1'b1`, latches `o_data <= parallel_data`, and reports whether a parity discrepancy occurred.

In all other FSM states, `o_valid`, `o_parity_err`, and `o_frame_err` are deasserted to low, providing a precise one-clock-cycle strobe and status handshake for downstream consumer logic.

### 3.6 Top-Level Receiver Integration

The top-level receiver module (`uart_rx`) combines the edge detector, FSM controller, deserializer, parity calculator, and output logic into the final RX module.

Incoming serial frames on `i_rx` are concurrently routed to the edge detector, the deserializer, and the output logic:

- `Start_Bit_Detect` triggers the `Controller` upon receiving the negative edge of the start bit.
- `Controller` shifts data into `Deserialiser` and tracks reception completion via `data_done`.
- `Parity` computes the expected parity of the incoming payload.
- `Output_Logic` samples the incoming parity and stop bits, checks for framing/parity violations, and provides registered data and valid pulses.
- The external `o_busy` flag directly reflects `ctrl_bus.o_busy`, indicating to host logic when a frame is currently in transit.